



Cinema Management Platform

Booking & Payment Services

Deep-Dive Architecture Design — Phase 2

Student Edition — Free-Tier Stack

|                   |                                                                                   |
|-------------------|-----------------------------------------------------------------------------------|
| Document Type     | Service Deep-Dive — Booking & Payment (Student Edition)                           |
| Edition           | Student Edition — Free-Tier Stack                                                 |
| Platform          | Cinema Management Platform                                                        |
| Phase             | 2 — Core Service Architecture                                                     |
| Constraint Source | Architecture Decision Summary (Phase 1 — Finalized)                               |
| Services In Scope | Booking Service · Payment Service                                                 |
| Focus Areas       | Concurrency · Consistency · Idempotency · Fault Tolerance                         |
| Cloud Target      | Docker Compose (local) · Supabase (PostgreSQL 15) · Upstash Redis · Upstash Kafka |
| Status            | Phase 2 Draft — Student Edition                                                   |

## Table of Contents

### 1. Service Responsibilities

- 1.1 Booking Service
- 1.2 Payment Service

### 2. Internal Module Design

- 2.1 Booking Service — Module Layout
- 2.2 Payment Service — Module Layout

### 3. Database Schema Design

- 3.1 Booking Service — DDL
- 3.2 Payment Service — DDL

### 4. Redis Strategy

- 4.1 Seat Lock Key Design
- 4.2 Atomic Lock Acquisition — Lua Script
- 4.3 Lock Release — Lua Script
- 4.4 Distributed Locking & Race Condition Prevention

### 5. API Design

- 5.1 Booking Service — REST Endpoints
- 5.1a Payment Service — REST Endpoints
- 5.2 Request / Response Payloads
- 5.3 Validation Strategy
- 5.4 Pagination & Filtering
- 5.5 Idempotency Key Strategy
- 5.6 Error Handling
- 5.7 Rate Limiting

### 6. Security Design

- 6.1 JWT Validation
- 6.2 RBAC Design
- 6.3 Payment Security
- 6.4 Payment Compliance Note (Student Edition)

### 7. Event Design

- 7.1 Kafka Topic Inventory
- 7.2 Event Contracts (Plain JSON — Jackson)
- 7.3 Retry Strategy
- 7.4 Dead Letter Queue Strategy

### 8. Distributed Transaction Design

- 8.1 Saga Pattern — Booking Confirmation Flow
- 8.2 Outbox Pattern Implementation
- 8.3 Compensation Handling
- 8.4 Eventual Consistency Handling

### 9. Sequence Diagrams (Mermaid.js)

- 9.1 Booking Workflow
- 9.2 Payment Workflow
- 9.3 Booking Cancellation Workflow

### 10. Failure Scenarios & Resiliency Handling

- 10.1 Redis Failure

- 10.1 Redis Failure
- 10.2 Database Failure
- 10.3 Kafka Failure
- 10.4 VNPay / Payment Gateway Failure
- 10.5 Network & Infrastructure Failures
- 10.6 Observability Integration for Failure Detection

## 11. Local Development Setup (Student Edition)

- 11.1 Prerequisites
- 11.2 Environment Variables (.env)
- 11.3 Docker Compose Services
- 11.4 ngrok Setup for VNPay IPN (Local Dev)

Cinema Management Platform · Student Edition · Free-Tier Stack

# 1. Service Responsibilities

## 1.1 Booking Service

The Booking Service is the core domain owner of seat reservation state. It is the sole authority on whether a seat is AVAILABLE, LOCKED, RESERVED, CONFIRMED, or CANCELLED and is the Saga initiator for the end-to-end ticket purchase flow.

### Primary Responsibilities

- Own the seat reservation state machine: AVAILABLE → LOCKED → RESERVED → CONFIRMED → CANCELLED
- Acquire and release distributed Redis seat locks (SETNX / 8-minute TTL)
- Validate showtime and hall configuration via synchronous REST call to Catalog service
- Persist booking aggregates and seat reservations to its own PostgreSQL instance (Supabase)
- Initiate the booking Saga by publishing to the booking.confirmed Kafka topic (Upstash Kafka)
- Handle Saga compensation: roll back seat state to AVAILABLE on payment.failed
- Implement the Transactional Outbox pattern to guarantee at-least-once Kafka delivery
- Push real-time seat map updates to WebSocket clients via STOMP
- Enforce idempotency on lock acquisition and booking creation
- Expose RBAC-gated REST endpoints to Spring Cloud Gateway

### Out of Scope

- Processing or validating payment card data — fully delegated to Payment Service / VNPay
- Sending email/push notifications — delegated to Notification Service via Kafka fan-out
- User authentication — JWT claims injected by Spring Cloud Gateway are trusted as-is

## 1.2 Payment Service

The Payment Service is the core domain owner of all financial transactions. It is a Kafka-only inbound service — it never accepts direct REST calls from Booking — and is responsible for idempotent VNPay payment URL generation and IPN processing.

### Primary Responsibilities

- Subscribe to the booking.confirmed Kafka topic for payment triggers
- Initiate VNPay payment URL generation and redirect the client to VNPay sandbox checkout
- Receive and verify VNPay IPN (Instant Payment Notification) callback via a dedicated REST endpoint
- Validate VNPay HMAC-SHA512 signature on every IPN request



- Persist payment records (status, VNPay transaction reference, amount) to its own PostgreSQL instance (Supabase)
- Publish payment.succeeded or payment.failed to Kafka on each terminal payment state
- Implement the Transactional Outbox pattern for guaranteed Kafka event delivery
- Enforce idempotency at the service level independent of Kafka delivery semantics
- Never expose financial data to other services except through published Kafka events

Out of Scope

- Seat state management — entirely owned by Booking Service
- Fraud scoring or advanced risk assessment — deferred to Phase 3 (VNPay handles baseline)
- Direct REST calls from any other service — Kafka is the only inbound trigger channel
- Raw payment card data — VNPay handles all card data on their hosted page (redirect flow)

Cinema Management Platform · Student Edition · Free-Tier Stack

2. Internal Module Design

2.1 Booking Service — Module Layout

| Layer          | Module / Package            | Responsibilities                                                                                                                        |
|----------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| API Layer      | booking.api.rest            | REST controllers, request/response DTOs, input validation (@Valid), OpenAPI annotations. Delegates to Application layer only.           |
| API Layer      | booking.api.websocket       | STOMP WebSocket handler. Pushes SeatStatusChangedEvent to /topic/showtimes/{id}/seats. Session routing via Upstash Redis.               |
| Application    | booking.application.command | Command handlers: LockSeatCommand, ConfirmBookingCommand, CancelBookingCommand. Orchestrates domain and infra.                          |
| Application    | booking.application.query   | Query handlers: GetBookingQuery, GetSeatMapQuery. Read-path; uses denormalized read models where applicable.                            |
| Application    | booking.application.saga    | Saga state tracker. Listens for payment.succeeded / payment.failed events and issues compensating commands.                             |
| Domain         | booking.domain.model        | Booking aggregate root, Seat value object, SeatStatus enum, BookingStatus enum, domain events (BookingCreated, SeatLocked, etc.).       |
| Domain         | booking.domain.service      | SeatLockDomainService: pure lock-acquisition logic. ShowtimeValidationService: validates capacity constraints.                          |
| Infrastructure | booking.infra.redis         | RedisSeatLockRepository: SETNX/GETDEL scripts. SeatMapCacheRepository: TTL-cached seat availability snapshots. Backed by Upstash Redis. |
| Infrastructure | booking.infra.persistence   | JPA repositories, Outbox table writer. Supabase connection pooler (Supavisor) config. Optimistic locking                                |

|                |                       |                                                                                                                      |
|----------------|-----------------------|----------------------------------------------------------------------------------------------------------------------|
|                |                       | on Booking entity.                                                                                                   |
| Infrastructure | booking.infra.kafka   | Outbox poller (Spring Scheduler). KafkaProducerConfig with JSON (Jackson) serializer. Upstash Kafka SASL/SSL config. |
| Infrastructure | booking.infra.catalog | Feign/RestTemplate client to Catalog service. Circuit breaker (Resilience4j). Response caching.                      |

2.2 Payment Service — Module Layout

| Layer          | Module / Package            | Responsibilities                                                                                                                                                      |
|----------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Application    | payment.application.command | ProcessPaymentCommand handler. Builds VNPAY payment URL, writes PENDING payment record and Outbox event.                                                              |
| Application    | payment.application.saga    | Listens for booking.cancellation_requested. Issues RefundCommand. Handles VNPAY refund result.                                                                        |
| Domain         | payment.domain.model        | Payment aggregate root, PaymentStatus enum (PENDING, SUCCEEDED, FAILED, REFUNDED). VNPAYTransaction value object.                                                     |
| Domain         | payment.domain.service      | IdempotencyChecker: queries existing payment by idempotency key. ChargePolicy: validates amount and currency.                                                         |
| Infrastructure | payment.infra.vnpay         | VNPAYGateway: builds VNPAY payment URL (HMAC-SHA512 signed). Verifies IPN callback signature. Maps VNPAY response codes to domain events.                             |
| Infrastructure | payment.infra.ipn           | VNPAYIPNController: REST endpoint POST /api/v1/payment/vnpay-ipn. Receives VNPAY server-to-server callback. Validates signature, then delegates to application layer. |
| Infrastructure | payment.infra.persistence   | JPA repositories, Outbox table writer. IdempotencyKey table for de-duplication. Supabase PostgreSQL connection via Spring Data JPA.                                   |
| Infrastructure | payment.infra.kafka         | KafkaConsumerConfig: booking.confirmed consumer group (Upstash Kafka). Outbox poller. JSON (Jackson) deserializer.                                                    |

3. Database Schema Design

CONSTRAINT

Each service owns its own logical schema within Supabase (PostgreSQL 15). No cross-service JOINS. Cross-service reads are via Kafka events or REST only — per the finalized Architecture Decision Summary.

3.1 Booking Service — DDL

```
--
-- BOOKING SERVICE DATABASE SCHEMA
-- PostgreSQL 15 (Supabase) | Partitioning by showtime_date | Optimistic locking
--

CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
```

```

-- -- Enum Types -----
CREATE TYPE seat_status AS ENUM ('AVAILABLE','LOCKED','RESERVED','CONFIRMED','CANCELLED');
CREATE TYPE booking_status AS ENUM
('INITIATED','CONFIRMED','PAYMENT_PENDING','COMPLETED','CANCELLED','FAILED');

-- -- bookings (RANGE-partitioned by showtime_date) -----
CREATE TABLE bookings (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID NOT NULL,
    showtime_id       UUID NOT NULL,
    cinema_id         UUID NOT NULL,          -- partition/routing key
    hall_id           UUID NOT NULL,
    showtime_date     DATE NOT NULL,         -- partition column
    status            booking_status NOT NULL DEFAULT 'INITIATED',
    total_amount      NUMERIC(10,2) NOT NULL CHECK (total_amount >= 0),
    currency          CHAR(3) NOT NULL DEFAULT 'SGD',
    idempotency_key   UUID NOT NULL,         -- client-supplied per attempt
    payment_ref       UUID,                 -- payment_id from Payment svc
    expires_at        TIMESTAMPTZ NOT NULL,  -- lock expiry (now + 8 min)
    confirmed_at      TIMESTAMPTZ,
    cancelled_at      TIMESTAMPTZ,
    cancellation_reason TEXT,
    version           BIGINT NOT NULL DEFAULT 0, -- optimistic lock
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (showtime_date);

-- Partitions managed manually or via pg_partman (future)

-- -- seat_reservations -----
CREATE TABLE seat_reservations (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id        UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    showtime_id       UUID NOT NULL,
    seat_id           UUID NOT NULL,
    row_label         VARCHAR(4) NOT NULL,
    seat_number       SMALLINT NOT NULL,
    seat_type         VARCHAR(32) NOT NULL, -- STANDARD, PREMIUM, VIP
    price             NUMERIC(10,2) NOT NULL,
    status            seat_status NOT NULL DEFAULT 'LOCKED',
    redis_lock_key    VARCHAR(255),         -- mirror of Redis key for audit
    locked_at         TIMESTAMPTZ,
    confirmed_at      TIMESTAMPTZ,
    released_at       TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- -- booking_outbox (Transactional Outbox pattern) -----
CREATE TABLE booking_outbox (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type    VARCHAR(64) NOT NULL, -- e.g. Booking
    aggregate_id      UUID NOT NULL,
    event_type        VARCHAR(128) NOT NULL, -- e.g. booking.confirmed
    payload           JSONB NOT NULL,
    kafka_topic       VARCHAR(255) NOT NULL,
    partition_key     VARCHAR(64) NOT NULL,  -- cinema_id for even partitioning
    status            VARCHAR(16) NOT NULL DEFAULT 'PENDING',
    attempt_count     SMALLINT NOT NULL DEFAULT 0,
    published_at      TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- -- booking_idempotency_keys -----
CREATE TABLE booking_idempotency_keys (
    idempotency_key   UUID PRIMARY KEY,
    booking_id        UUID NOT NULL REFERENCES bookings(id),
    user_id           UUID NOT NULL,
    request_hash      VARCHAR(64) NOT NULL,
    response_status    SMALLINT NOT NULL,

```

```

        response_body    JSONB,
        created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        expires_at       TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '24 hours'
    );

-- -- INDEXES -----
CREATE UNIQUE INDEX ux_bookings_idempotency ON bookings(idempotency_key);
CREATE INDEX ix_bookings_user_id ON bookings(user_id);
CREATE INDEX ix_bookings_showtime_id ON bookings(showtime_id);
CREATE INDEX ix_bookings_cinema_id ON bookings(cinema_id);
CREATE INDEX ix_bookings_status ON bookings(status)
    WHERE status NOT IN ('COMPLETED', 'CANCELLED');
CREATE INDEX ix_bookings_expires_at ON bookings(expires_at)
    WHERE status = 'INITIATED';

CREATE UNIQUE INDEX ux_seat_res_showtime_seat ON seat_reservations(showtime_id, seat_id)
    WHERE status NOT IN ('CANCELLED');
CREATE INDEX ix_seat_res_booking_id ON seat_reservations(booking_id);
CREATE INDEX ix_seat_res_showtime_status ON seat_reservations(showtime_id, status);

CREATE INDEX ix_outbox_status_created ON booking_outbox(status, created_at)
    WHERE status = 'PENDING';

-- -- TRIGGERS -----
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bookings_updated_at
    BEFORE UPDATE ON bookings
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

```

### 3.2 Payment Service — DDL

```

-- -----
-- PAYMENT SERVICE DATABASE SCHEMA
-- PostgreSQL 15 (Supabase) | Separate logical schema | VNPay fields
-- -----

CREATE TYPE payment_status AS ENUM
    ('PENDING', 'PROCESSING', 'SUCCEEDED', 'FAILED', 'REFUNDED', 'PARTIALLY_REFUNDED');
CREATE TYPE payment_gateway AS ENUM ('VNPAY');

-- -- payments -----
CREATE TABLE payments (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id        UUID NOT NULL,          -- FK to Booking svc (logical, no constraint)
    user_id           UUID NOT NULL,
    cinema_id         UUID NOT NULL,
    idempotency_key   VARCHAR(255) NOT NULL, -- booking_id::attempt_number
    gateway            payment_gateway NOT NULL DEFAULT 'VNPAY',
    status            payment_status NOT NULL DEFAULT 'PENDING',
    amount            NUMERIC(10,2) NOT NULL CHECK (amount > 0),
    currency          CHAR(3) NOT NULL DEFAULT 'SGD',
    refunded_amount   NUMERIC(10,2) NOT NULL DEFAULT 0,
    vnpay_txn_ref     VARCHAR(255),           -- vnp_TxnRef
    vnpay_transaction_no VARCHAR(255),       -- vnp_TransactionNo
    vnpay_response_code VARCHAR(10),         -- vnp_ResponseCode from IPN
    vnpay_bank_code   VARCHAR(20),           -- Issuing bank code from VNPay
    failure_code      VARCHAR(128),
    failure_message    TEXT,
    metadata          JSONB,
    processed_at      TIMESTAMPTZ,
    refunded_at       TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```
-- -- payment_outbox -----
CREATE TABLE payment_outbox (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type VARCHAR(64) NOT NULL,
  aggregate_id  UUID NOT NULL,
  event_type    VARCHAR(128) NOT NULL,
  payload       JSONB NOT NULL,
  kafka_topic   VARCHAR(255) NOT NULL,
  partition_key VARCHAR(64) NOT NULL,
  status        VARCHAR(16) NOT NULL DEFAULT 'PENDING',
  attempt_count SMALLINT NOT NULL DEFAULT 0,
  published_at  TIMESTAMPTZ,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- -- payment_idempotency_keys -----
CREATE TABLE payment_idempotency_keys (
  idempotency_key VARCHAR(255) PRIMARY KEY,
  payment_id       UUID NOT NULL REFERENCES payments(id),
  booking_id       UUID NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at       TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '48 hours'
);

-- -- INDEXES -----
CREATE UNIQUE INDEX ux_payments_idempotency ON payments(idempotency_key);
CREATE UNIQUE INDEX ux_payments_booking_id ON payments(booking_id)
  WHERE status NOT IN ('FAILED', 'REFUNDED');
CREATE INDEX ix_payments_vnpay_txn ON payments(vnpay_txn_ref);
CREATE INDEX ix_payments_user_id ON payments(user_id);
CREATE INDEX ix_payments_status ON payments(status)
  WHERE status IN ('PENDING', 'PROCESSING');
CREATE INDEX ix_payments_cinema_created ON payments(cinema_id, created_at DESC);
CREATE INDEX ix_outbox_pay_status ON payment_outbox(status, created_at)
  WHERE status = 'PENDING';
```

4. Redis Strategy

**ARCHITECTURE CONSTRAINT**  
Upstash Redis (free tier) is the seat-lock source of truth per the finalized ADR. A seat is available if and only if no Redis lock exists. The seat map UI cache may lag — the POST /bookings/lock endpoint always reads Redis directly, never the UI cache.

4.1 Seat Lock Key Design

| Key Pattern                       | Value                             | TTL                                       |
|-----------------------------------|-----------------------------------|-------------------------------------------|
| seat:lock:{showtime_id}:{seat_id} | booking_id (UUID string)          | 480s (8 minutes — per ADR)                |
| seat:map:{showtime_id}            | JSON map {seat_id: status}        | 60s (cache snapshot; refreshed on change) |
| booking:session:{booking_id}      | JSON {userId, seats[], expiresAt} | 490s (10s grace over lock TTL)            |
| ws:session:{session_id}           | pod_id (for STOMP routing)        | 3600s (WebSocket session lifetime)        |
| rate:lock:{user_id}               | attempt count (INCR)              | 60s (sliding window for rate limiting)    |

Note (Student Edition): Upstash Redis free tier supports up to 10,000 commands/day and 256 MB storage. Seat lock TTL (480s) and map cache (60s) are well within limits for demo/staging traffic.

4.2 Atomic Lock Acquisition — Lua Script

**CONCURRENCY CONTROL**  
All multi-key seat lock operations use a single Lua script executed atomically on the Redis server. This eliminates race conditions between check-and-set steps and is the primary mechanism for double-booking prevention.

```
-- Lua script: acquire N seat locks atomically (EVALSHA)
-- KEYS: seat:lock:{showtime_id}:{seat_id} for each requested seat
-- ARGV[1]: booking_id  ARGV[2]: ttl_seconds  ARGV[3..N]: seat_ids (for audit)

local booking_id = ARGV[1]
local ttl = tonumber(ARGV[2])
local acquired = {}

-- Phase 1: validate all seats are free (no partial locks)
for i = 1, #KEYS do
    local existing = redis.call("GET", KEYS[i])
    if existing and existing ~= booking_id then
        for _, k in ipairs(acquired) do redis.call("DEL", k) end
        return {0, KEYS[i]} -- 0 = failure, KEYS[i] = conflicting seat
    end
end

-- Phase 2: atomically SETNX all seats
for i = 1, #KEYS do
    local ok = redis.call("SET", KEYS[i], booking_id, "NX", "EX", ttl)
    if not ok then
        for _, k in ipairs(acquired) do redis.call("DEL", k) end
        return {0, KEYS[i]}
    end
    table.insert(acquired, KEYS[i])
end

return {1} -- 1 = all locks acquired successfully
```

4.3 Lock Release — Lua Script

```
-- Lua script: release seat locks only if owned by this booking_id
-- Prevents accidental release of another booking's locks after TTL expiry + re-lock

local booking_id = ARGV[1]
local released = 0

for i = 1, #KEYS do
    local owner = redis.call("GET", KEYS[i])
    if owner == booking_id then
        redis.call("DEL", KEYS[i])
        released = released + 1
    end
end

return released
```

4.4 Distributed Locking & Race Condition Prevention

| Scenario                                    | Prevention Mechanism                                                                                            |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Two users select the same seat concurrently | Atomic Lua SETNX script: only one writer wins. The other receives HTTP 409 CONFLICT with seat_id in error body. |
| Booking expires before payment completes    | TTL 480s. If the payment succeeded must expire within 480s. If                                                  |

5. API Design

5.1 Booking Service — REST Endpoints

| Method | Path                         | Description                                                            | Auth / RBAC                     |
|--------|------------------------------|------------------------------------------------------------------------|---------------------------------|
| POST   | /api/v1/bookings/lock        | Acquire Redis seat locks. Idempotent via Idempotency-Key header.       | JWT — ROLE_CUSTOMER             |
| POST   | /api/v1/bookings             | Confirm booking after lock. Initiates Saga (writes to Outbox).         | JWT — ROLE_CUSTOMER             |
| GET    | /api/v1/bookings/{id}        | Get booking detail. User can only see own bookings; admin sees all.    | JWT — ROLE_CUSTOMER, ROLE_ADMIN |
| GET    | /api/v1/bookings             | List user's bookings. Cursor-based pagination. Filter by status, date. | JWT — ROLE_CUSTOMER             |
| DELETE | /api/v1/bookings/{id}        | Cancel booking. Triggers compensating Saga. Returns 202 Accepted.      | JWT — ROLE_CUSTOMER, ROLE_ADMIN |
| GET    | /api/v1/showtimes/{id}/seats | Get live seat map. Reads Redis snapshot; falls back to DB.             | JWT — any authenticated         |
| GET    | /api/v1/bookings/{id}/ticket | Download e-ticket PDF/QR. Only for COMPLETED bookings.                 | JWT — ROLE_CUSTOMER (own)       |

5.1a Payment Service — REST Endpoints

Note: VNPay IPN endpoint must be publicly accessible (VNPay calls it server-to-server). In local dev, use ngrok or similar tunneling tool to expose the endpoint.

| Method | Path                      | Description                                                                                                                                  | Auth / RBAC                                              |
|--------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| POST   | /api/v1/payments/initiate | Triggered internally after booking.confirmed consumed. Generates VNPay payment URL. Returns redirect URL to client via WebSocket or polling. | Internal (no direct client call)                         |
| POST   | /api/v1/payment/vnpay-ipn | VNPay IPN callback endpoint. Server-to-server only. Validates HMAC-SHA512 signature. Delegates to application layer.                         | No JWT — IP whitelist (VNPay IPs) + signature validation |

|     |                                     | signature. Delegates to application layer.                                                       | validation                |
|-----|-------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------|
| GET | /api/v1/payments/{bookingId}/status | Poll payment status for a given booking. Used by client while waiting for VNPay redirect return. | JWT — ROLE_CUSTOMER (own) |

## 5.2 Request / Response Payloads

### POST /api/v1/bookings/lock — Request

```

Headers:
  Authorization: Bearer <jwt>
  Idempotency-Key: <uuid> // Required; client-generated UUID per attempt
  Content-Type: application/json

Body:
{
  "showtime_id": "uuid",
  "seat_ids": ["uuid-1", "uuid-2"], // 1-8 seats per booking
  "cinema_id": "uuid" // Used as Kafka partition key
}

```

### POST /api/v1/bookings/lock — Response (201 Created)

```

{
  "booking_id": "uuid",
  "status": "LOCKED",
  "locked_seats": [
    { "seat_id": "uuid-1", "row": "C", "number": 12, "type": "PREMIUM", "price": 18.50 },
    { "seat_id": "uuid-2", "row": "C", "number": 13, "type": "PREMIUM", "price": 18.50 }
  ],
  "total_amount": 37.00,
  "currency": "SGD",
  "expires_at": "2025-01-15T14:08:00Z", // now + 8 minutes
  "lock_ttl_seconds": 480
}

Error 409 Conflict (seat already locked):
{
  "error": "SEAT_CONFLICT",
  "message": "One or more seats are no longer available",
  "conflicting_seats": ["uuid-1"],
  "trace_id": "abc123"
}

```

### POST /api/v1/bookings — Request (Confirm booking)

```

{
  "booking_id": "uuid", // from lock step
  "return_url": "https://yourapp.com/booking/result",
  "user_email": "user@example.com",
  "user_name": "Jane Doe"
}

```

### POST /api/v1/payments/initiate — Response (200 OK)

```

{
  "payment_url": "https://sandbox.vnpayment.vn/paymentv2/vpcpay.html?vnp_Amount=...",
  "booking_id": "uuid",
  "expires_at": "2025-01-15T14:08:00Z",
  "vnp_txn_ref": "uuid-truncated-15chars"
}

```

### VNPay IPN Callback — Incoming (from VNPay server)



```
{
  "vnp_Amount":      "3700000",
  "vnp_BankCode":    "NCB",
  "vnp_BankTranNo":  "VNP...",
  "vnp_CardType":     "ATM",
  "vnp_OrderInfo":    "Thanh toan booking uuid",
  "vnp_PayDate":      "20250115140800",
  "vnp_ResponseCode": "00",
  "vnp_TmnCode":      "YOUR_TMN_CODE",
  "vnp_TransactionNo": "14057285",
  "vnp_TransactionStatus": "00",
  "vnp_TxnRef":       "booking-uuid-ref",
  "vnp_SecureHash":   "hmac_sha512_signature"
}

VNPay IPN Response (Payment Service → VNPay):
{"RspCode": "00", "Message": "Confirm Success"}
```

5.3 Validation Strategy

| Field                   | Rule                                                                 | Implementation                                                                  |
|-------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| seat_ids                | 1–8 items; all UUIDs; all belong to showtime_id                      | Bean Validation @Size + custom @SeatsBelongToShowtime constraint                |
| showtime_id / cinema_id | Must exist in Catalog; not in the past                               | Catalog client call at lock time; cached for 30s                                |
| Idempotency-Key header  | Required for POST /lock and POST /bookings; UUID format              | @NotNull + UUID pattern regex in controller advice                              |
| return_url              | Valid HTTPS URL; non-empty                                           | Regex validation; used as vnp_ReturnUrl in VNPay URL builder                    |
| booking_id on confirm   | Must match an INITIATED booking owned by the requesting user         | DB lookup + JWT sub claim comparison in application layer                       |
| vnp_SecureHash          | HMAC-SHA512 computed with VNPay secret key; must match incoming hash | Computed server-side in VNPayGateway.verifyIpnSignature() before any processing |
| vnp_ResponseCode        | Must equal "00" for success                                          | Checked in ProcessPaymentCommand handler                                        |
| Request body size       | Max 16 KB                                                            | Spring Cloud Gateway request size filter                                        |

5.4 Pagination & Filtering

All collection endpoints use cursor-based pagination (not offset) to ensure consistent results under high write load.

```
GET /api/v1/bookings?
  cursor=<opaque_base64_cursor> // encodes: {created_at, id} of last seen record
  limit=20                       // default 20, max 100
  status=COMPLETED,CANCELLED   // comma-separated enum values
  from_date=2025-01-01
  to_date=2025-03-31
  cinema_id=<uuid>               // ADMIN only filter

Response:
{
  "items":      [...],
  "next_cursor": "eyJjcmVhdGVkX2F0IjoiMjAyNS0wMS0xNVQxMjowMDowMFoiLCJpZCI6InVlaWQifQ==",
  "has_more":   true,
  "total_count": null // omitted for performance; use Prometheus metrics for counts
}
```

5.5 Idempotency Key Strategy

| Aspect | Design Decision                                                                                      |
|--------|------------------------------------------------------------------------------------------------------|
| Scope  | Required on all POST endpoints that mutate state (POST /lock, POST /bookings, DELETE /bookings/{id}) |
| Format | Client-generated UUID v4 in Idempotency-Key HTTP header                                              |

|                    |                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| Format             | Client-generated UUID v4 in Idempotency-Key HTTP header                                                                      |
| Storage            | Stored in booking_idempotency_keys table with response body (JSONB) and TTL 24h                                              |
| Matching           | On repeat request with same key: return stored response with HTTP 200 (not 201) and X-Idempotent-Replay: true header         |
| Conflict detection | Same key + different request body hash → HTTP 422 with IDEMPOTENCY_KEY_REUSE error                                           |
| Expiry             | Keys expire after 24h; clients must generate new key for retries beyond 24h                                                  |
| Payment service    | Uses composite key: booking_id::attempt_number embedded in the Kafka event payload; stored in payment_idempotency_keys table |

5.6 Error Handling

| HTTP Status | Error Code             | Scenario                                                                                  |
|-------------|------------------------|-------------------------------------------------------------------------------------------|
| 400         | VALIDATION_ERROR       | Invalid request body, missing required fields, invalid UUID format                        |
| 401         | UNAUTHORIZED           | Missing or invalid JWT. Spring Cloud Gateway returns this before request reaches service. |
| 403         | FORBIDDEN              | JWT valid but insufficient role, or accessing another user's booking                      |
| 404         | BOOKING_NOT_FOUND      | booking_id or showtime_id does not exist                                                  |
| 409         | SEAT_CONFLICT          | One or more requested seats are already locked or reserved                                |
| 409         | BOOKING_STATE_CONFLICT | Attempt to cancel a COMPLETED booking or confirm an EXPIRED one                           |
| 422         | IDEMPOTENCY_KEY_REUSE  | Same idempotency key sent with different request payload                                  |
| 422         | SHOWTIME_EXPIRED       | Showtime is in the past                                                                   |
| 429         | RATE_LIMIT_EXCEEDED    | Too many booking attempts. X-RateLimit-Reset header included.                             |
| 500         | INTERNAL_ERROR         | Unexpected server error. trace_id always included for correlation.                        |
| 503         | SERVICE_UNAVAILABLE    | Redis or DB unavailable. Circuit breaker open. Retry-After header set.                    |

```
// Standard error response envelope
{
  "error": "SEAT_CONFLICT",
  "message": "Seats [uuid-1, uuid-2] are no longer available",
  "details": {
    "conflicting_seats": ["uuid-1", "uuid-2"]
  },
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "timestamp": "2025-01-15T14:00:05.123Z"
}
```

5.7 Rate Limiting

| Endpoint / Scope                   | Limit                                          | Implementation                                                                                         |
|------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| POST /bookings/lock (per user)     | 5 requests / 60 seconds                        | Spring Cloud Gateway RequestRateLimiter filter (backed by Upstash Redis); X-RateLimit-Remaining header |
| POST /bookings (per user)          | 3 requests / 60 seconds                        | Spring Cloud Gateway RequestRateLimiter filter                                                         |
| GET /showtimes/{id}/seats (per IP) | 120 requests / 60 seconds                      | Spring Cloud Gateway RequestRateLimiter filter; higher for anonymous browsing                          |
| DELETE /bookings/{id} (per user)   | 10 requests / 60 seconds                       | Spring Cloud Gateway RequestRateLimiter filter                                                         |
| Global (per API key/IP)            | 2,000 requests / 60 seconds                    | Spring Cloud Gateway global route filter                                                               |
| Burst protection                   | Token bucket; burst allowance = 2× steady rate | Spring Cloud Gateway token bucket via RedisRateLimiter bean                                            |

## 6. Security Design

### 6.1 JWT Validation

| Aspect           | Design                                                                                                                                                                             |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Algorithm        | RS256 (RSA-2048 asymmetric). Identity service holds private key (environment variable / .env file). Services only need the public key.                                             |
| Key distribution | JWKS endpoint on Identity service. Spring Cloud Gateway caches JWKS locally (5-minute TTL). Services never call Identity service per request.                                      |
| Claims used      | sub (user_id), roles[], cinema_id (for operator scoping), exp, iat, jti                                                                                                            |
| Propagation      | Spring Cloud Gateway validates JWT via JWKS, then injects X-User-Id and X-User-Roles headers. Downstream services trust these headers only — never re-parse the JWT.               |
| Token expiry     | 15-minute access token; 7-day refresh token. Refresh handled entirely by Identity service.                                                                                         |
| Revocation       | Short-lived tokens reduce revocation surface. Compromised tokens handled via jti deny-list in Upstash Redis (Identity service publishes revocations to token.revoked Kafka topic). |

### 6.2 RBAC Design

| Role          | Permissions                                                                | Enforcement Point                                                          |
|---------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------|
| ROLE_CUSTOMER | POST /lock, POST /bookings, GET own bookings, DELETE own booking           | Spring Security @PreAuthorize on service methods; user_id claim comparison |
| ROLE_OPERATOR | GET all bookings for owned cinema_id, manage showtimes/pricing via Catalog | cinema_id claim in JWT compared to requested resource cinema_id            |
| ROLE_ADMIN    | GET all bookings, cancel any booking, access analytics endpoints           | ROLE_ADMIN in X-User-Roles header; checked in Spring Security filter chain |

### 6.3 Payment Security

| Control                  | Implementation                                                                                                                                                                                                                                                  |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Card data isolation      | Raw card data never touches the Cinema platform. VNPay redirect flow handles all card input on VNPay's hosted page. Cinema platform only receives a transaction reference and status.                                                                           |
| Payment scope            | Platform is fully out of PCI-DSS scope for card data. VNPay is the PCI-certified payment processor.                                                                                                                                                             |
| IPN signature validation | All VNPay IPN callbacks validated with HMAC-SHA512 using vnp_HashSecret (stored in .env / Docker Compose environment variable). Invalid signatures → HTTP 200 {"RspCode": "97", "Message": "Invalid Checksum"} returned to VNPay (do not expose error details). |
| Replay attack prevention | vnp_TxnRef stored and checked for duplicate IPN delivery. Duplicate IPN → return {"RspCode": "02", "Message": "Order already confirmed"}.                                                                                                                       |
| IPN endpoint security    | Endpoint accessible without JWT (VNPay calls server-to-server). Protected by: (1) HMAC-SHA512 signature validation, (2) VNPay IP whitelist in Spring Cloud Gateway route config, (3) idempotency guard on vnp_TxnRef.                                           |
| Credential storage       | VNPay vnp_TmnCode and vnp_HashSecret stored in .env file (never committed to                                                                                                                                                                                    |

7. Event Design

7.1 Kafka Topic Inventory

| Topic               | Partitions | Retention | Partition Key  |
|---------------------|------------|-----------|----------------|
| booking.seat-locked | 8          | 7 days    | cinema_id      |
| booking.confirmed   | 8          | 7 days    | cinema_id      |
| booking.cancelled   | 8          | 7 days    | cinema_id      |
| booking.completed   | 8          | 7 days    | cinema_id      |
| booking.expired     | 8          | 7 days    | cinema_id      |
| payment.initiated   | 8          | 7 days    | cinema_id      |
| payment.succeeded   | 8          | 7 days    | cinema_id      |
| payment.failed      | 8          | 7 days    | cinema_id      |
| payment.refunded    | 8          | 7 days    | cinema_id      |
| booking.dlq         | 4          | 30 days   | original_topic |
| payment.dlq         | 4          | 30 days   | original_topic |

7.2 Event Contracts (Plain JSON — Jackson)

Events are serialized as plain JSON (Jackson ObjectMapper). Events use plain JSON only — no external schema management tooling is required. Event schema versioning is handled via the schema\_version field in each payload envelope.

booking.confirmed — Event Payload

```
{
  "schema_version": "1.0",
  "event_id": "uuid",
  "event_type": "booking.confirmed",
  "trace_id": "opentelemetry-trace-id",
  "occurred_at": "ISO-8601 timestamp",
  "aggregate_id": "booking-uuid",
  "aggregate_type": "Booking",
  "payload": {
    "booking_id": "uuid",
    "user id": "uuid",
```

```

    "cinema_id":      "uuid",
    "showtime_id":    "uuid",
    "total_amount":   "37.00",
    "currency":        "SGD",
    "return_url":      "https://yourapp.com/booking/result",
    "attempt_number": 1,
    "seats": [
      { "seat_id": "uuid", "row": "C", "number": 12, "type": "PREMIUM", "price": "18.50" }
    ],
    "idempotency_key": "booking-uuid:1"
  }
}

```

### payment.succeeded — Event Payload

```

{
  "schema_version": "1.0",
  "event_id":        "uuid",
  "event_type":       "payment.succeeded",
  "trace_id":         "opentelemetry-trace-id",
  "occurred_at":      "ISO-8601 timestamp",
  "payload": {
    "payment_id":      "uuid",
    "booking_id":       "uuid",
    "user_id":          "uuid",
    "cinema_id":        "uuid",
    "amount":           "37.00",
    "currency":         "SGD",
    "gateway":          "VNPAY",
    "vnpay_txn_ref":    "booking-uuid-ref",
    "vnpay_transaction_no": "14057285",
    "vnpay_bank_code":  "NCB",
    "processed_at":     "ISO-8601 timestamp"
  }
}

```

## 7.3 Retry Strategy

| Aspect                   | Design                                                                                                                                                                          |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consumer retry policy    | Spring Kafka DefaultErrorHandler: 3 attempts with exponential backoff (1s, 2s, 4s). Backoff capped at 30s for transient DB/Redis errors.                                        |
| Non-retryable exceptions | JsonProcessingException (Jackson parse error), EntityNotFoundException → sent directly to DLQ without retry (DeadLetterPublishingRecoverer).                                    |
| Retryable exceptions     | DataAccessException, RedisConnectionException, VNPay HTTP 5xx → retry with backoff.                                                                                             |
| Outbox poller retry      | Spring Scheduler runs every 5s. Outbox records with attempt_count > 5 are marked FAILED and generate an alert. Separate DLQ reprocessing job via Admin UI.                      |
| Saga timeout retry       | If payment.succeeded not received within 9 minutes (lock TTL + 1 min grace), Booking service marks booking FAILED and publishes booking.expired. Separate Spring Scheduler job. |

## 7.4 Dead Letter Queue Strategy

| Aspect            | Design                                                                                                                         |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------|
| DLQ topics        | booking.dlq and payment.dlq. Partition key = original_topic for consumer grouping.                                             |
| DLQ record format | Original message + DLQ headers: X-Original-Topic, X-Exception-Class, X-Exception-Message, X-Attempt-Count, X-Failed-At         |
| DLQ consumer      | Separate consumer group (booking-dlq-consumer) running at reduced concurrency. Processes with additional logging and alerting. |
| Monitoring        | Prometheus alert fires if DLQ consumer lag > 0 for more than 5 minutes. Alert routed to on-call engineer.                      |

|               |                                                                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Reprocessing  | Admin UI exposes DLQ replay endpoint (POST /admin/dlq/replay).<br>Replays to original topic with incremented attempt number. Requires ROLE_ADMIN. |
| DLQ retention | 30 days (vs 7 days for normal topics). Allows investigation window for incidents.                                                                 |

Cinema Management Platform · Student Edition · Free-Tier Stack

## 8. Distributed Transaction Design

### 8.1 Saga Pattern — Booking Confirmation Flow

|                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PATTERN</b><br>Choreography-based Saga (per finalized ADR). No central orchestrator. Services react to Kafka events and publish compensating events. This aligns with the Payment service being Kafka-only for inbound triggers. |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| Step             | Service        | Action                                                                                                                                                                                                                                                                                                       | On Failure                                                                                                                                                                                                     |
|------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1                | Booking        | Acquire Redis seat locks (Lua SETNX). Write INITIATED booking to DB.                                                                                                                                                                                                                                         | Release any partial locks via rollback Lua script. Return 409 to client.                                                                                                                                       |
| 2                | Booking        | Confirm booking: write DB record, write Outbox event (booking.confirmed). Both in same DB transaction.                                                                                                                                                                                                       | DB transaction rollback. Release Redis locks. Return 500 to client.                                                                                                                                            |
| 3                | Booking Outbox | Poller reads PENDING Outbox rows; publishes to Upstash Kafka booking.confirmed.                                                                                                                                                                                                                              | Retry with backoff. Outbox ensures at-least-once delivery.                                                                                                                                                     |
| 4                | Payment        | Consume booking.confirmed. Check idempotency key. Generate VNPay payment URL (HMAC-SHA512 signed). Write PENDING payment to DB. Push payment URL to client (via polling GET /payments/{bookingId}/status). Await VNPay IPN callback. On IPN receipt: validate signature → write PENDING → SUCCEEDED payment. | Write payment.failed. VNPay IPN with ResponseCode != "00" treated as payment failure. DLQ on persistent Kafka failure. Invalid IPN signature → reject silently (return error code to VNPay, do not update DB). |
| 5                | Payment Outbox | Publish payment.succeeded or payment.failed to Kafka.                                                                                                                                                                                                                                                        | Retry with backoff.                                                                                                                                                                                            |
| 6                | Booking        | Consume payment.succeeded: update booking to COMPLETED. Update seat_reservations to CONFIRMED. Release Redis locks.                                                                                                                                                                                          | payment.failed: trigger compensation (Step 7).                                                                                                                                                                 |
| 7 (compensation) | Booking        | On payment.failed: update booking to FAILED. Update seats to AVAILABLE. Release Redis locks. Publish booking.cancelled.                                                                                                                                                                                      | Idempotent compensation via version check.                                                                                                                                                                     |
| 8                | Notification   | Consume booking.completed or booking.cancelled. Send email/push to user.                                                                                                                                                                                                                                     | DLQ for notification. Does not affect booking state.                                                                                                                                                           |

### 8.2 Outbox Pattern Implementation

The Transactional Outbox pattern guarantees that domain state changes and Kafka event publishing are always consistent, even if the Upstash Kafka broker is temporarily unavailable.

|                                                      |
|------------------------------------------------------|
| // Booking Service – within a single DB transaction: |
|------------------------------------------------------|

```
@Transactional
public BookingResponse confirmBooking(ConfirmBookingCommand cmd) {
    // 1. Update booking entity (optimistic lock via @Version)
    Booking booking = bookingRepo.findById(cmd.bookingId())
        .orElseThrow(() -> new BookingNotFoundException(cmd.bookingId()));
    booking.confirm(); // state machine transition; throws if invalid state

    // 2. Write to Outbox table – same transaction as booking update
    OutboxEvent event = OutboxEvent.builder()
        .aggregateType("Booking")
        .aggregateId(booking.getId())
        .eventType("booking.confirmed")
        .kafkaTopic("booking.confirmed")
        .partitionKey(booking.getCinemaId().toString())
        .payload(eventMapper.toBookingConfirmedPayload(booking, cmd))
        .build();
    outboxRepo.save(event);

    // 3. Both writes commit atomically to Supabase. If Upstash Kafka is down, Outbox row persists.
    return bookingMapper.toResponse(booking);
}

// Outbox Poller (separate Spring Scheduler thread):
@Scheduled(fixedDelay = 5000)
public void pollAndPublish() {
    List<OutboxEvent> pending = outboxRepo.findTop100ByStatusOrderByCreatedAtAsc("PENDING");
    for (OutboxEvent event : pending) {
        try {
            kafkaTemplate.send(event.getKafkaTopic(),
                               event.getPartitionKey(),
                               JsonSerializer.serialize(event.getPayload()))
                .get(5, TimeUnit.SECONDS); // synchronous confirm
            outboxRepo.markSent(event.getId(), Instant.now());
        } catch (Exception e) {
            outboxRepo.incrementAttempt(event.getId());
        }
    }
}
```

8.3 Compensation Handling

| Compensation Trigger                         | Compensating Action                                                                                                         | Idempotency Guard                                                   |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| payment.failed received                      | Booking → FAILED; seats → AVAILABLE; Redis locks deleted; booking.cancelled published                                       | Booking version check; only compensates if status = PAYMENT_PENDING |
| Booking lock TTL expired (8 min)             | Spring Scheduler job queries expired INITIATED bookings; moves to EXPIRED; releases any lingering locks                     | Processed with DB-level SELECT FOR UPDATE SKIP LOCKED               |
| VNPay refund on cancellation                 | Payment svc calls VNPay refund API; updates payment to REFUNDED; publishes payment.refunded                                 | Check payment.status = SUCCEEDED before initiating refund           |
| Saga timeout (payment not received in 9 min) | Booking service publishes booking.expired; moves booking to FAILED; Payment service checks for orphaned PROCESSING payments | Processed by separate timeout scheduler with idempotency key check  |
| Partial seat lock failure                    | Lua script rolls back all acquired locks atomically before returning failure                                                | Atomic rollback in single Lua execution — no partial state possible |

8.4 Eventual Consistency Handling

| Consistency Concern            | Handling Strategy                                                                                                                                                               |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seat map UI lag vs Redis truth | UI reads from seat:map:{showtime_id} Redis cache (60s TTL). WebSocket STOMP push updates UI within ~1s on lock state change. Client shows "Refreshing..." spinner on reconnect. |
| Analytics read model lag       | Acceptable; Analytics is CQRS read model. Projections are eventually consistent with transactional state. SLA: < 5 minutes behind.                                              |

9. Sequence Diagrams (Mermaid.js)

The following Mermaid.js diagrams can be rendered at mermaid.live or embedded in any documentation system supporting Mermaid rendering (Confluence, GitLab, Notion, GitHub Markdown).

9.1 Booking Workflow

Sequence Diagram (Mermaid.js)

```
sequenceDiagram
    autonumber
    actor Client
    participant GW as Spring Cloud Gateway
    participant BS as Booking Service
    participant Redis as Redis (Upstash)
    participant DB_B as Booking DB (Supabase)
    participant Kafka as Kafka (Upstash)
    participant Cat as Catalog Service

    Client->>GW: POST /api/v1/bookings/lock (JWT + Idempotency-Key)
    GW->>GW: Validate JWT (JWKS cache). Inject X-User-Id, X-User-Roles
    GW->>BS: Forward request
    BS->>DB_B: Check idempotency key (SELECT)
    alt Idempotency key exists
        DB_B-->>BS: Cached response
        BS-->>GW: 200 + X-Idempotent-Replay: true
        GW-->>Client: 200 (cached)
    end
    BS->>Cat: GET /showtimes/{id} (validate, check capacity)
    Cat-->>BS: ShowtimeDetails (cached)
    BS->>Redis: EVALSHA lua_acquire_locks [seat_ids] booking_id TTL=480
    alt All seats free
        Redis-->>BS: {1} (success)
    else Seat conflict
        Redis-->>BS: {0, conflicting_seat_id}
        BS-->>GW: 409 SEAT_CONFLICT
        GW-->>Client: 409
    end
    BS->>DB_B: BEGIN TX: INSERT booking (INITIATED) + INSERT outbox(booking.seat-locked)
    DB_B-->>BS: COMMIT
    BS-->>GW: 201 {booking_id, locked_seats, expires_at}
    GW-->>Client: 201

    Client->>GW: POST /api/v1/bookings (confirm, return_url)
    GW->>BS: Forward
```



```

BS->>DB_B: BEGIN TX: UPDATE booking -> CONFIRMED + INSERT outbox(booking.confirmed)
DB_B-->>BS: COMMIT
BS-->>-GW: 202 Accepted (async - payment pending)
GW-->>-Client: 202
BS->>+Kafka: Outbox poller publishes booking.confirmed
Kafka-->>-BS: ack
note over Client,Kafka: Payment flow continues (see Diagram 9.2)

```

## 9.2 Payment Workflow

### Sequence Diagram (Mermaid.js)

```

sequenceDiagram
    autonumber
    participant Kafka as Kafka (Upstash)
    participant PS as Payment Service
    participant DB_P as Payment DB (Supabase)
    participant VNP as VNPay Sandbox
    participant BS as Booking Service
    participant Client as Client (Browser)
    participant Notif as Notification Service

    Kafka->>+PS: booking.confirmed event
    PS->>DB_P: SELECT idempotency_key = booking_id::attempt_number
    alt Idempotency key exists
        DB_P-->>PS: Existing payment record
        PS->>Kafka: ACK (no-op)
        PS-->>-Kafka: Done
    end
    PS->>DB_P: BEGIN TX - INSERT payment(PENDING) + INSERT outbox
    DB_P-->>PS: COMMIT
    PS->>PS: Build VNPay payment URL
        (HMAC-SHA512 signed with vnp_HashSecret)
    PS-->>Client: Return payment_url (via polling GET /payments/{bookingId}/status)
    Client->>+VNP: Browser redirect to VNPay payment page
    Client->>VNP: User enters card details and confirms
    VNP-->>-Client: Redirect to return_url (display result)

    VNP-->>+PS: POST /api/v1/payment/vnpay-ipn (server-to-server)
    PS->>PS: Validate HMAC-SHA512 signature
    alt Signature invalid
        PS-->>VNP: {"RspCode":"97","Message":"Invalid Checksum"}
    end
    PS->>DB_P: Check vnp_TxnRef for duplicate IPN
    alt Duplicate IPN
        PS-->>VNP: {"RspCode":"02","Message":"Order already confirmed"}
    end
    alt vnp_ResponseCode = "00" (success)
        PS->>DB_P: BEGIN TX - UPDATE payment->SUCCEEDED + outbox(payment.succeeded)
        DB_P-->>PS: COMMIT
        PS->>Kafka: Outbox poller publishes payment.succeeded
        PS-->>-VNP: {"RspCode":"00","Message":"Confirm Success"}
    else Payment failed
        PS->>DB_P: BEGIN TX - UPDATE payment->FAILED + outbox(payment.failed)
        DB_P-->>PS: COMMIT
        PS->>Kafka: Outbox poller publishes payment.failed
        PS-->>-VNP: {"RspCode":"00","Message":"Confirm Success"}
    end

    Kafka->>+BS: payment.succeeded
    BS->>BS: Update booking->COMPLETED. Update seat_reservations->CONFIRMED.
    BS->>+Redis: EVALSHA lua_release_locks [seat_ids] booking_id
    Redis-->>-BS: released count
    BS->>Kafka: Publish booking.completed
    BS-->>-Kafka: ACK

    Kafka->>+Notif: booking.completed (fan-out consumer group)
    Notif->>Notif: Send email + push notification
    Notif-->>-Kafka: ACK

```

## 9.3 Booking Cancellation Workflow

### Sequence Diagram (Mermaid.js)

```
sequenceDiagram
    autonumber
    actor Client
    participant GW as Spring Cloud Gateway
    participant BS as Booking Service
    participant DB_B as Booking DB (Supabase)
    participant Redis as Redis (Upstash)
    participant Kafka as Kafka (Upstash)
    participant PS as Payment Service
    participant VNP as VNPay Sandbox
    participant Notif as Notification Service

    Client->>GW: DELETE /api/v1/bookings/{id} (JWT)
    GW->>GW: Validate JWT. Check ROLE_CUSTOMER or ROLE_ADMIN.
    GW->>BS: Forward
    BS->>DB_B: SELECT booking FOR UPDATE (check ownership, check status = COMPLETED)
    alt Booking not COMPLETED
        DB_B-->>BS: Booking found, not COMPLETED
        BS-->>GW: 409 BOOKING_STATE_CONFLICT
        GW-->>Client: 409
    end
    BS->>DB_B: BEGIN TX: UPDATE booking -> CANCELLATION_REQUESTED
    BS->>DB_B: INSERT outbox(booking.cancellation_requested)
    DB_B-->>BS: COMMIT
    BS-->>GW: 202 Accepted
    GW-->>Client: 202
    BS->>Kafka: Outbox poller publishes booking.cancellation_requested
    Kafka->>PS: booking.cancellation_requested
    PS->>DB_P: SELECT payment WHERE booking_id = ? AND status = SUCCEEDED
    PS->>VNP: CreateRefund(vnpay_transaction_no, amount, idempotency_key)
    VNP-->>PS: Refund(status: succeeded)
    PS->>DB_P: BEGIN TX: UPDATE payment -> REFUNDED + INSERT outbox(payment.refunded)
    DB_P-->>PS: COMMIT
    PS->>Kafka: Outbox poller publishes payment.refunded
    PS-->>Kafka: ACK
    Kafka->>BS: payment.refunded
    BS->>DB_B: BEGIN TX: UPDATE booking -> CANCELLED. Update seat_reservations -> AVAILABLE.
    BS->>Redis: EVALSHA lua_release_locks (idempotent - locks may already be TTL-expired)
    BS->>DB_B: INSERT outbox(booking.cancelled)
    DB_B-->>BS: COMMIT
    BS->>Kafka: Publish booking.cancelled
    BS-->>Kafka: ACK
    Kafka->>Notif: booking.cancelled (fan-out)
    Notif->>Notif: Send cancellation + refund email
    Notif-->>Kafka: ACK
```

Cinema Management Platform · Student Edition · Free-Tier Stack

## 10. Failure Scenarios & Resiliency Handling

### 10.1 Redis Failure

**10.1 Redis Failure**

| Failure Mode                      | Resiliency Response                                                                                                                                                                 |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Redis node failure (primary)      | Upstash Redis (managed, auto-replicated): automatic failover. Booking service uses Resilience4j CircuitBreaker on Redis calls. Circuit OPEN → return HTTP 503 with Retry-After: 60. |
| Redis connection timeout on SETNX | Lettuce connection pool with configurable timeout (default 200ms). Timeout → treat as lock failure (conservative). Return 503 to client.                                            |
| All Redis locks lost on failover  | Upstash Redis handles replication internally. For local Docker Compose dev: single Redis container; document as known single-point-of-failure acceptable for demo environment.      |
| Redis partition / split-brain     | Single Upstash Redis instance (free tier). Upgrade path to Upstash Redis cluster documented for production phase.                                                                   |

**10.2 Database Failure**

| Failure Mode                                | Resiliency Response                                                                                                                                                                          |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Supabase primary failure                    | Supabase manages PostgreSQL availability. Connection pooler (Supavisor, built-in) absorbs connection churn. Active connections fail with retrievable error. HA deferred to production phase. |
| Optimistic lock conflict (version mismatch) | Spring Retry: 3 attempts with 50ms backoff on OptimisticLockingFailureException. Persistent conflict → 409 BOOKING_STATE_CONFLICT returned to client.                                        |
| Outbox poller DB timeout                    | Outbox records remain PENDING. Poller retries on next 5s cycle. No data loss — events are not lost, just delayed. Alert if PENDING records age > 60s.                                        |
| DB connection pool exhaustion               | Supabase Supavisor connection pooler. HPA on Booking pods triggered by CPU/memory before pool exhaustion at peak.                                                                            |

**10.3 Kafka Failure**

| Failure Mode                   | Resiliency Response                                                                                                                                                                                  |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Broker unavailable on publish  | Outbox pattern: DB write commits independently of Kafka publish. Outbox poller retries publish until broker is available. No data loss — events queued in Outbox table.                              |
| Consumer group rebalance       | Kafka consumer commits offsets after successful DB write and Outbox update. Rebalance → uncommitted offset replayed. Idempotency keys prevent duplicate processing.                                  |
| Upstash Kafka broker failure   | Upstash manages broker availability and replication. Consumer lag monitored via Prometheus JMX exporter. Auto-scales Booking pods when lag > 1000 messages.                                          |
| Message schema incompatibility | Plain JSON (Jackson): schema versioning via schema_version field. Consumers apply lenient deserialization (unknown fields ignored). Breaking changes handled via version negotiation in the payload. |
| Slow consumer / consumer lag   | Spring Boot Actuator + Prometheus metrics. Auto-scale Booking pods up to max 20 replicas when lag > 1000 messages.                                                                                   |

**10.4 VNPay / Payment Gateway Failure**

| Failure Mode                              | Resiliency Response                                                                                                                                                                                   |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VNPay sandbox unavailable                 | Payment URL generation fails → publish payment.failed → Saga compensation. User sees "Payment gateway unavailable, please retry."                                                                     |
| IPN not received (VNPay delivery failure) | VNPay retries IPN for up to 3 attempts. If still not received: client polling GET /payments/{bookingId}/status returns PENDING; booking expires after 9-minute Saga timeout → FAILED, seats released. |
| IPN signature invalid                     | Reject silently. Return {"RspCode":"97"} to VNPay. Do not update DB. Log with trace_id for investigation.                                                                                             |
| Duplicate IPN delivery                    | Idempotency guard on vnp_TxnRef: second IPN returns {"RspCode":"02"} without reprocessing.                                                                                                            |
| VNPay response code != "00"               | Treat as payment failure. Publish payment.failed. Saga compensation: booking→FAILED, seats→AVAILABLE.                                                                                                 |

|                               |                                                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| ngrok tunnel down (local dev) | IPN cannot reach local server. Use VNPay sandbox "Query Transaction" API to manually verify status during development. |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------|

10.5 Network & Infrastructure Failures

| Failure Mode                                   | Resiliency Response                                                                                                                                                                     |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Booking → Catalog service timeout              | Resilience4j CircuitBreaker + Timeout. Fallback: return cached showtime data (max 5 minutes stale). If no cache: reject booking with 503.                                               |
| Docker Compose container crash during booking  | Docker Compose restart: unless-stopped policy. Outbox ensures in-flight events are not lost. Clients should use Idempotency-Key for retry — idempotent response returned on re-attempt. |
| Single Docker host (local dev)                 | Single Docker host (local dev). Note as known limitation: no multi-AZ redundancy in dev environment. HA deferred to production phase.                                                   |
| API Gateway (Spring Cloud Gateway) pod failure | Spring Cloud Gateway restart via Docker Compose restart policy. Health probes configured. Configuration stored in application.yml (version-controlled).                                 |

10.6 Observability Integration for Failure Detection

| Signal                        | Implementation                                                                                                                                                            |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Booking failure rate alert    | Prometheus alert: booking_confirmation_errors_total / booking_confirmation_total > 0.01 for 2 minutes → notification to on-call                                           |
| Payment failure rate alert    | payment_failed_total rate > baseline_stddev × 3 for 5 minutes → P1 alert                                                                                                  |
| Redis lock latency alert      | redis_seat_lock_duration_p95 > 500ms → P2 alert                                                                                                                           |
| Outbox stuck alert            | outbox_pending_age_seconds > 60 → Slack #alerts channel warning; > 300s → P1 alert                                                                                        |
| Kafka consumer lag alert      | kafka_consumer_lag{group="booking-service"} > 1000 → autoscale trigger + Slack alert                                                                                      |
| Distributed trace correlation | OpenTelemetry trace_id propagated through all Kafka events (as header), Redis keys (as value suffix), and DB records. Full trace visible in Tempo for any failed booking. |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student Edition Observability Stack (Docker Compose):                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <ul style="list-style-type: none"><li>• Metrics: Prometheus (scrapes all Spring Boot Actuator /actuator/prometheus endpoints)</li><li>• Dashboards: Grafana (pre-built Spring Boot + JVM dashboards via dashboard provisioning)</li><li>• Logs: Loki + Promtail (structured JSON logs from all services)</li><li>• Traces: Tempo + OpenTelemetry Java Agent (auto-instrumentation; trace_id propagated through Kafka headers and HTTP)</li><li>• All components run as Docker Compose services; no cloud dependency required</li></ul> |

11. Local Development Setup (Student Edition)

11.1 Prerequisites

- Java 21 (Eclipse Temurin recommended)
- Docker Desktop (Docker Compose v2)
- Maven 3.9+
- ngrok (for VNPay IPN local testing)

11.2 Environment Variables (.env)

Create a .env file in the project root (never commit to Git):

```
# Supabase
SUPABASE_BOOKING_DB_URL=jdbc:postgresql://db.<project>.supabase.co:5432/postgres
SUPABASE_BOOKING_DB_USER=postgres
SUPABASE_BOOKING_DB_PASSWORD=<your-password>
SUPABASE_PAYMENT_DB_URL=jdbc:postgresql://db.<project>.supabase.co:5432/postgres

# Upstash Kafka
UPSTASH_KAFKA_BOOTSTRAP=<cluster>.upstash.io:9092
UPSTASH_KAFKA_USERNAME=<username>
UPSTASH_KAFKA_PASSWORD=<password>

# Upstash Redis
UPSTASH_REDIS_URL=rediss://:<password>@<cluster>.upstash.io:6379

# VNPay Sandbox
VNPAY_TMN_CODE=<your-tmn-code>
VNPAY_HASH_SECRET=<your-hash-secret>
VNPAY_URL=https://sandbox.vnpayment.vn/paymentv2/vpcpay.html
VNPAY_RETURN_URL=http://localhost:8080/booking/result

# JWT
JWT_PRIVATE_KEY_PATH=./keys/private.pem
JWT_PUBLIC_KEY_PATH=./keys/public.pem
```

11.3 Docker Compose Services

| Service          | Component                | Port              |
|------------------|--------------------------|-------------------|
| gateway          | Spring Cloud Gateway     | :8080             |
| identity-service | Identity Service         | :8081             |
| catalog-service  | Catalog Service          | :8082             |
| booking-service  | Booking Service          | :8083             |
| payment-service  | Payment Service          | :8084             |
| notification-svc | Notification Service     | :8085             |
| prometheus       | Prometheus               | :9090             |
| grafana          | Grafana                  | :3000             |
| loki             | Loki                     | :3100             |
| promtail         | Promtail (log collector) | (no exposed port) |
| tempo            | Tempo (trace backend)    | :3200             |

External (cloud free tier):

Supabase

PostgreSQL per service

(cloud)

Upstash Kafka

Kafka topics

(cloud)

Upstash Redis

Seat locks + rate limiting

(cloud)

11.4 ngrok Setup for VNPay IPN (Local Dev)

```
# 1. Start ngrok tunnel pointing at Payment Service port
ngrok http 8084

# 2. Copy the HTTPS forwarding URL, e.g.:
# https://abc123.ngrok.io
```

```
# 3. Set in .env:  
VNPAY_IPN_URL=https://abc123.ngrok.io/api/v1/payment/vnpay-ipn  
  
# 4. Update VNPay merchant portal sandbox settings with the same IPN URL  
  
# Note: ngrok free tier URL changes on each restart.  
#       Update VNPAY_IPN_URL and the merchant portal setting on each dev session.
```

Cinema Management Platform · Student Edition · Free-Tier Stack